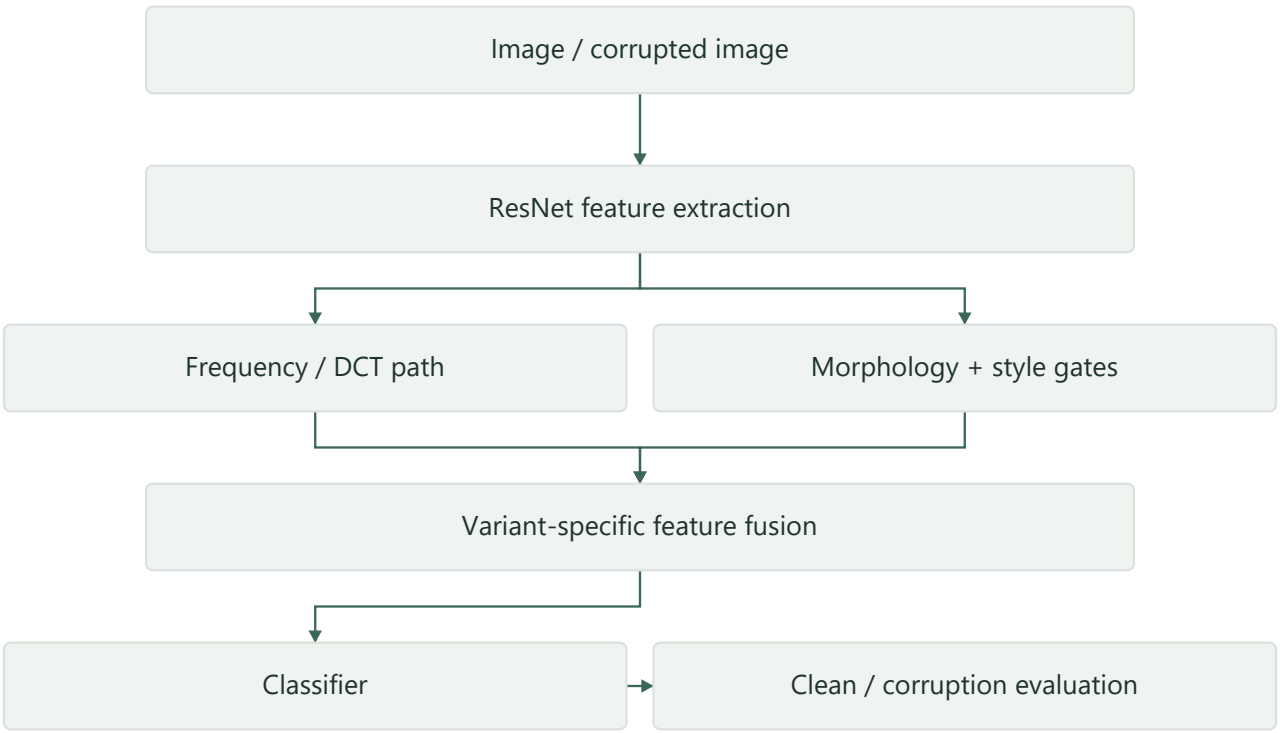
QFM Research

PhaseForge · frequency and morphology in CNNs

A family of convolutional-network experiments combining frequency transforms, morphology-related gates, and configurable robustness evaluations.

Architecture



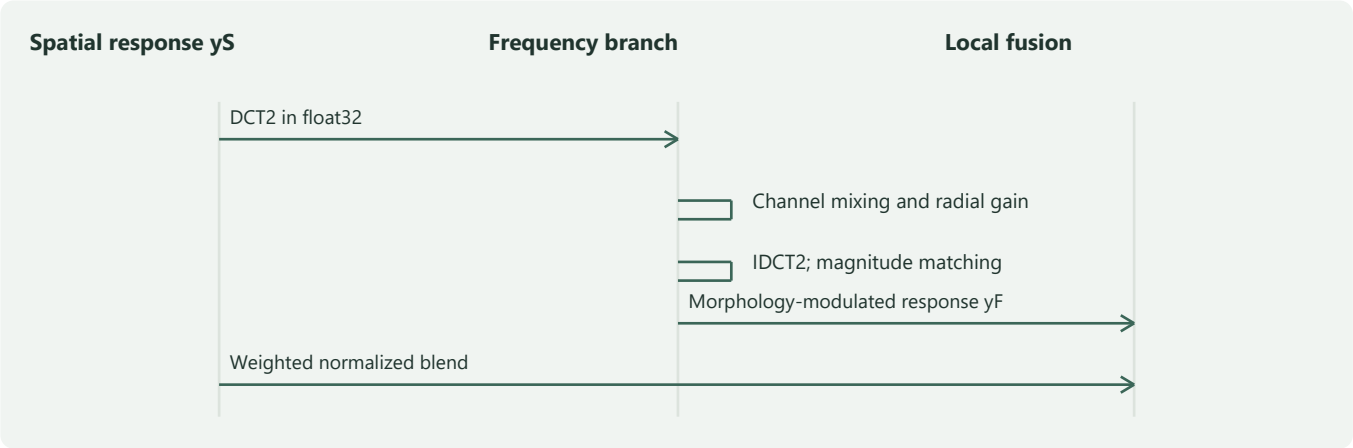
Component overview derived from the supplied source; version-specific details may differ.

Component	Implementation	Role
Model components	QFM-DCT modules	Frequency processing and gates
Backbone	ResNet-18 integration	Configurable replacement blocks
Training	CIFAR wrapper and run configs	Clean and augmented environments
Evaluation	Baseline and ablation runs	Noise, JPEG, blur, low-light
Versions	qfmcnn / QFM_Research / robust-qfm	Distinct experimental implementations

Research implementation

Inside the QFM-DCT block

This chapter focuses on the modular QFM_Research implementation. Its frequency branch transforms the spatial response of a convolutional block, rather than treating the entire network as a single frequency transform. The source keeps transform calculations in float32 even when surrounding training uses mixed precision.



Transform and parameterization

For one channel, the separable transform has the form X = C_H x C_W^T and the inverse x = C_H^T X C_W, using orthonormal DCT matrices cached in float32. The transformed response is mixed across channels with learned weights. A radial grid is constructed in DCT index space, and a smooth band applies different low- and high-frequency gains.

Numerical controls

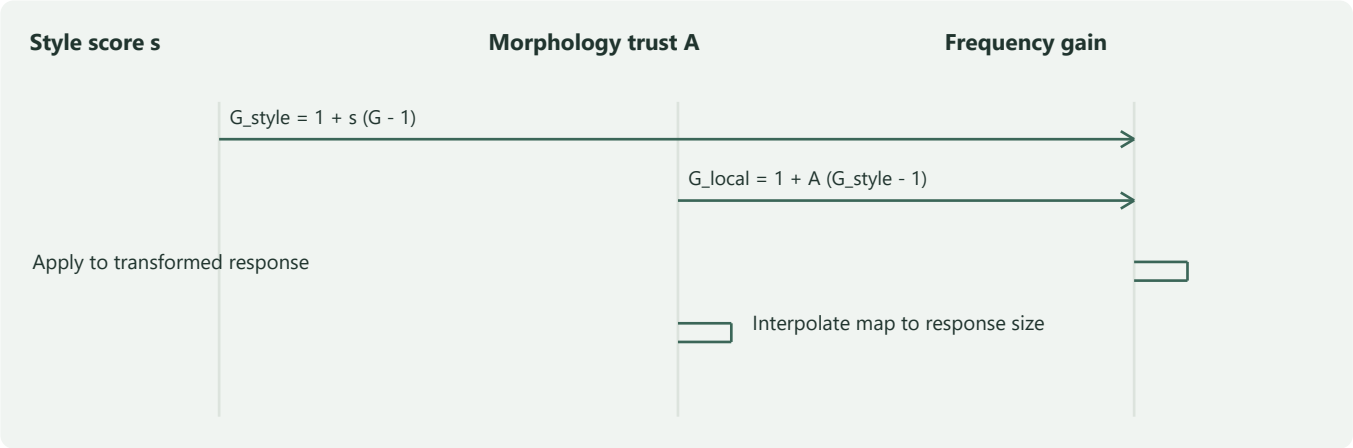
In forward(), the radial threshold is squashed into [0.20,0.55], the low gain into [0.70,1.00], and the high gain into [1.00,1.30]. Band steepness is clamped to [1,40]. The inverse-transformed branch is rescaled by the ratio of spatial mean absolute magnitude to frequency-branch magnitude plus 1e-6. These controls constrain scale; they do not by themselves demonstrate robustness.

Tensor stage	Shape / operation
Spatial convolution	B x C_out x H_out x W_out
Frequency channel mix	einsum oc,bchw->bohw; spatial frequency grid retained
Reconstruction	IDCT and dtype restoration before branch blending

Source / interpretation: QFM_Research/qfm_dct.py: dct2, idct2 and QFMConv2dDCT.forward. Shape diagram is specific to this inspected modular branch.

Gates, style and ablations

The block uses multiple gates with distinct roles. A morphology-derived local map controls trust; image-level style proxies alter the gain strength and branch balance. Explaining them separately makes it possible to ask which mechanism contributes to a result instead of attributing every change to the QFM name.



Local and global controls

The source derives style proxies for noise, JPEG blockiness, blur, low light and high-frequency loss. Style pushes the gain toward identity for lower scores. The morphology map is interpolated to the frequency grid and clamped into [0,1] for the trust blend. A separate morphology modulation also acts on the reconstructed branch, with a clipped multiplicative range of 0.5 to 1.5.

Fusion and controlled comparisons

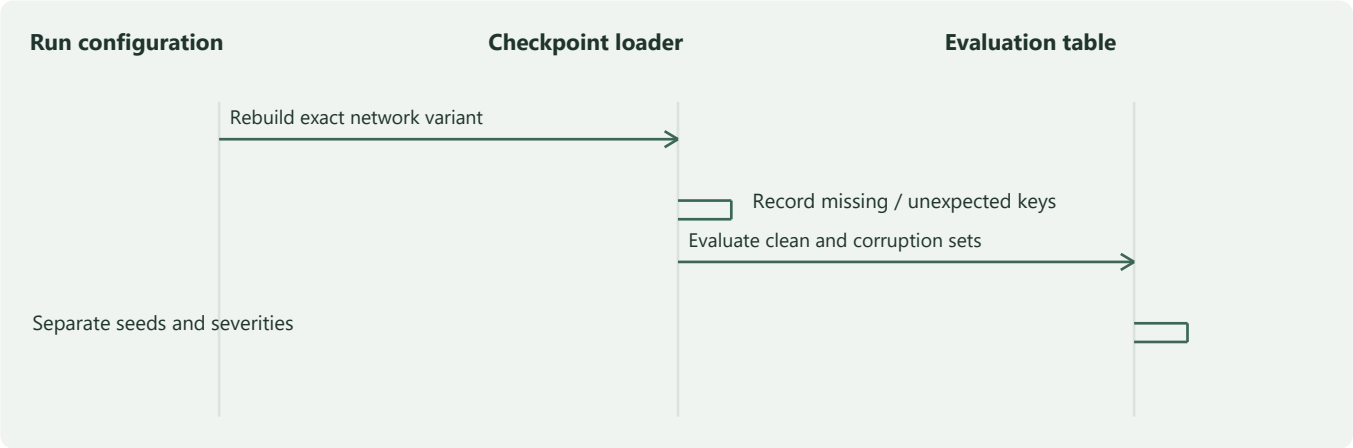
The final response divides the weighted sum of spatial and frequency branches by their summed weights, with a small denominator floor. Disabling frequency, morphology, style or phase is a useful controlled comparison only when the remaining run settings are aligned. The source includes a Cayley-based mixer for the spatial filter bank; its presence is a parameterization choice, not evidence of execution on quantum hardware.

Ablation	Question	Control to retain
Frequency off	Does transformed processing help?	Backbone, split, seed and training budget
Morphology off	Does local trust matter?	Frequency settings and style policy
Style off	Does input-conditioned gating help?	Corruption definitions and severity

Source / interpretation: QFM_Research/qfm_dct.py: SimpleMorphGate, OrthoMixer, style_score and forward gating. These are methodological comparisons, not newly measured ablation results.

Versioning & evaluation protocol

Three source families are retained because their differences can affect model behavior: original qfmcnn demonstrations, modular QFM_Research, and robust-qfm training. A portfolio should explain that progression while attaching each reported figure to one exact definition and checkpoint.



Checkpoint compatibility

The evaluator accepts several checkpoint dictionary formats and reports missing or unexpected state keys rather than enforcing an exact match in every path. A run can therefore proceed without establishing that every intended parameter loaded. Reproduction should preserve those diagnostics and verify architecture switches, replacement blocks, input resolution and class count.

Reading robustness results

Noise, JPEG, blur and low-light environments are named in the local training/evaluation code. A fair comparison needs identical evaluation inputs, documented severity, multiple seeds and consistent compute budgets. Clean accuracy and corruption accuracy should both be reported so a robustness change is not detached from its clean-data tradeoff. No comparative advantage is inferred here while the Drive run archive remains unreconciled.

Family	Role	Keep distinct
qfmcnn	Earlier training, demos, export and media	Original checkpoint and diagram versions
QFM_Research	Modular DCT blocks and ablation evaluation	Configuration and replacement rules
robust-qfm	Robustness-oriented trainer and gate handling	Stride placement and interpolation choices

Source / interpretation: QFM_Research/eval_robustness.py, train_cifar.py, Tests/test_qfm_shapes.py; robust-qfm/qfmcnnreftrain.py; qfmcnn source inventory.

Evidence & source

Implementation, supporting records, and the boundary of the claim

Area	Evidence	Reference
Source family	20 source candidates across three folders	Local source inventory
Experiment controls	Frequency / morphology / style / phase switches	QFM model and configuration files
Outcome status	Comparative results pending reconciliation	Drive experiment archive awaited

Selected implementation files

File / record	Path / reference
qfm_dct.py	QFM_Research/qfm_dct.py
eval_robustness.py	QFM_Research/eval_robustness.py
test_qfm_shapes.py	QFM_Research/Tests/test_qfm_shapes.py

Selected model and evaluation modules. Version-specific checkpoints and run manifests will accompany the research archive.

Evidence boundary

The diagram is a module-level view across variants, not a tensor-exact graph of one frozen network. No baseline improvement or quantum-hardware execution is claimed from the project name or source alone.

Technical note

The modular version includes DCT/IDCT processing, cached transform matrices, a Cayley-based mixer, and per-sample style features. The robustness variant changes where transforms operate relative to stride.